

Notebook Construction and Audit Strategies

This section outlines two complementary frameworks for working with computational notebooks in machine learning and data science contexts: a *Construction Framework* for authoring new notebooks, and an *Audit Framework* for formally reviewing completed drafts. While iterative evaluation is a natural part of notebook construction, applying a formal audit too early can disrupt the exploratory flow. Therefore, these frameworks are applied sequentially within a conditional feedback loop, illustrated in Figure 1, where audit findings drive subsequent construction revisions.

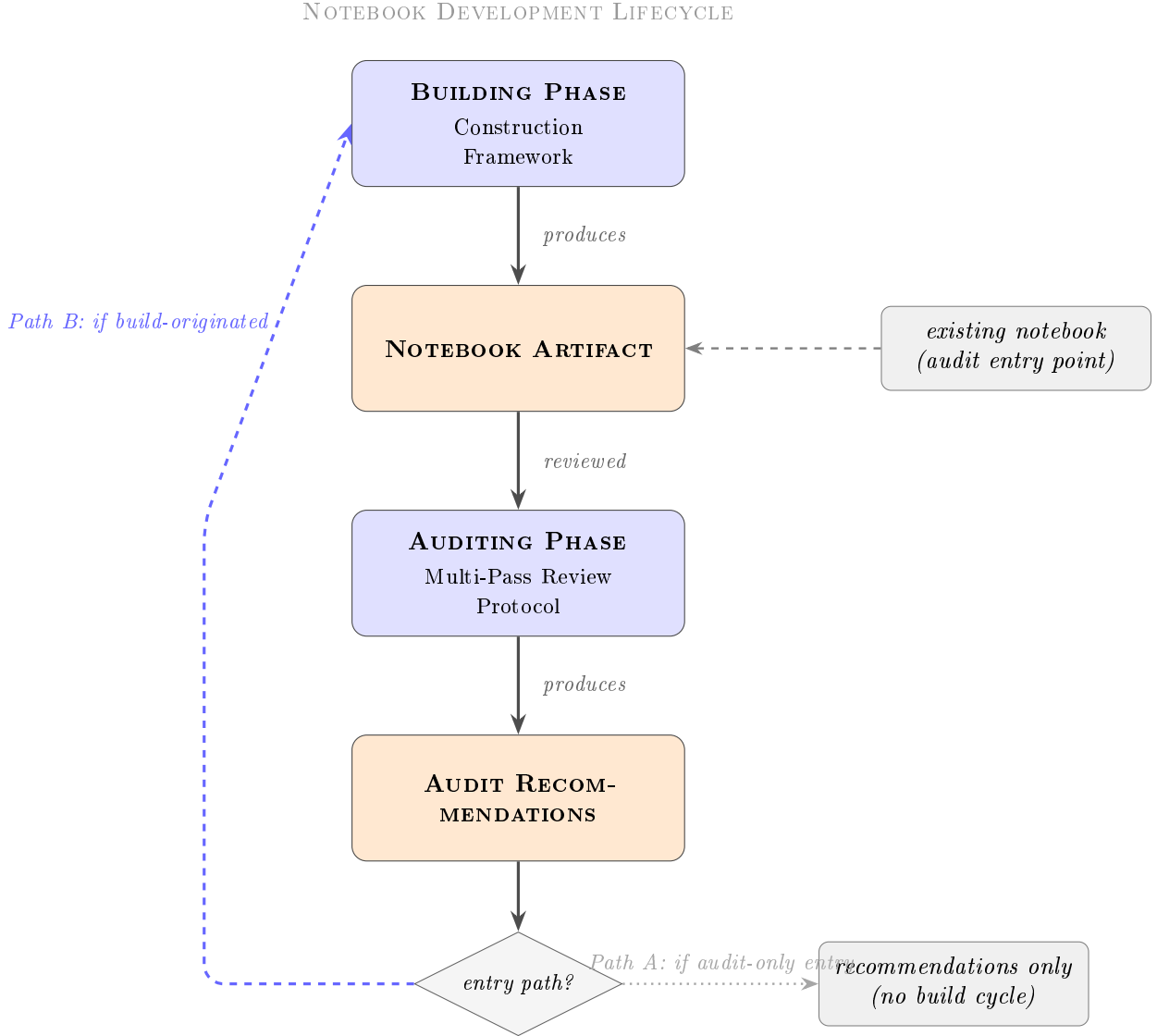


Figure 1: The iterative lifecycle of a computational notebook. Initial drafting and exploratory evaluation occur under the Construction Framework (Part I) before transitioning to the formal Audit Framework (Part II). Recommendations then resolve through one of two mutually exclusive paths depending on entry point: Path B closes the loop back into the Building Phase for notebooks that originated there; Path A terminates at the recommendations for notebooks that entered directly via the audit-only route, unless subsequently routed into construction.

Part I — Notebook Construction Framework

The Construction Framework is a *forward-looking prescription*: it guides the notebook author in producing a well-structured, reproducible, and maintainable artifact from the ground up. It is organized into three sequential phases.

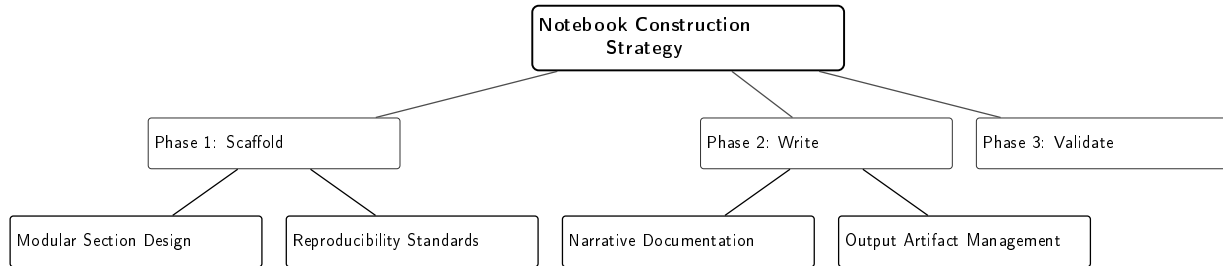


Figure 2: High-level pillars of the Notebook Construction Framework.

Phase 1 — Scaffold: Establish the structural and reproducibility foundation before writing any logic:

- Define the notebook’s **single responsibility** — one notebook should correspond to one coherent workflow or experiment.
- Create standard section headers as markdown cells, following this canonical order:
 1. Environment & Dependencies
 2. Configuration & Global Parameters
 3. Data Ingestion & Schema Validation
 4. Data Partitioning & Split Manifest
 5. Preprocessing & Feature Engineering
 6. Model Definition & Training
 7. Evaluation & Metrics
 8. Artifact Export (models, plots, reports)
 9. Conclusions & Next Steps

Data Partitioning and Split-Manifest Contract When data partitioning is applicable, the *Data Partitioning & Split Manifest* section shall define the split strategy, split keys, random or chronological boundaries, grouping constraints, stratification rules, and the immutable assignment of observations or groups to the required partitions before any fitted preprocessing, feature selection, model fitting, threshold selection, hyperparameter selection, or model selection is performed.

The Split Manifest shall be bound to the immutable identity of the exact source-data snapshot from which the partition assignment was generated. At minimum, it shall record:

- the dataset name and immutable version, release identifier, registry identifier, or equivalent snapshot reference;

- the integrity reference for every local input file or for the versioned dataset manifest governing a collection of input files;
- the applicable schema or data-contract version;
- the expected observation or group identifier set, represented directly or through a verified integrity reference;
- the stable observation or group identifier used for partitioning;
- the partition assigned to each observation or group, either directly or through a versioned assignment artifact;
- the partitioning method and its relevant configuration;
- the random seeds, temporal boundaries, grouping keys, stratification variables, filtering rules, and exclusion criteria used to generate the assignment; and
- the integrity reference for the completed Split Manifest and any linked assignment artifact.

A split assignment shall be valid only for the exact dataset snapshot, schema, observation set, filtering and deduplication rules, and partitioning configuration recorded in the Split Manifest. Any material change to those elements shall invalidate the previous Split Manifest and require generation of a new manifest before preprocessing, training, or evaluation continues.

Before the split is consumed, the implementation shall verify that the observed dataset identity matches the recorded source-data identity, that each expected observation or group is assigned consistently, that no prohibited overlap exists across partitions, and that all declared grouping, chronological, and stratification constraints are satisfied.

Phase 2 — Write: Author each section incrementally, following these discipline rules:

- Work through **one section at a time** (Divide and Conquer principle).
- Accompany every code cell with a markdown explanation of intent and expected output.
- Avoid burying logic inside loops or helper calls without commentary.
- Prefer **explicit variable passing** over hidden or global state to maintain cell independence.
- Summarize repetitive code patterns with a general description and highlight meaningful variations. Three or more similar blocks shall be treated as a refactoring candidate. Extracted logic may be moved into a local Python module only when that module is versioned, included in the project repository, listed in the Audit Package Manifest, and reviewed together with the notebook.
- Route all persisted outputs (models, plots, reports) through a single, versioned export convention defined at the start of the section — never write artifacts ad hoc from arbitrary cells.

Phase 3 — Validate During Writing: Continuously verify correctness as each section is completed while maintaining a strict separation between partial-development executions and complete release-bearing runs.

Execution-Attempt and Release-Run Contract Every checkpoint, exploratory, or partial execution shall receive a unique *Execution Attempt Identifier*. Its execution record shall identify:

- the governing manifest versions;
- the notebook section or workflow stage executed;
- the execution status;
- the artifacts produced; and
- whether the execution was partial or complete.

Artifacts produced by a partial execution shall be stored in a development namespace associated with the Execution Attempt Identifier. They shall not be written to, promoted into, or interpreted as artifacts of a complete release run.

A release-bearing *Run Identifier* shall be used only for a fresh-kernel, end-to-end execution of the complete applicable workflow. Its lifecycle shall use the following states:

`COMPLETE_UNVERIFIED` and `COMPLETE_VERIFIED`.

A complete run shall remain `COMPLETE_UNVERIFIED` until all required section contracts, data constraints, expected outputs, and artifact-integrity checks have been verified. Only then may it transition to `COMPLETE_VERIFIED` and support release, publication, or deployment.

- After finalizing a section, restart the kernel and execute all cells up to that section under a new Execution Attempt Identifier.
- Verify that the section consumes only declared inputs, produces the documented outputs and side effects, and does not depend on execution of later cells or undeclared mutable state.
- Evaluate section-level reproducibility using the applicable deterministic checks or pre-defined stochastic tolerances.
- Store all checkpoint outputs under the corresponding Execution Attempt Identifier. Partial execution shall never overwrite, replace, or verify a complete-run artifact.
- Resolve execution failures, manifest mismatches, undeclared state, partition inconsistencies, or missing expected outputs before proceeding.
- After all applicable sections are complete, restart the kernel and execute the notebook end-to-end under the release Run Identifier.
- Verify the complete artifact set before marking the run `COMPLETE_VERIFIED`.
- Re-execution shall create a new execution-attempt record. It may verify an existing release artifact set when the governing run identity is unchanged, but it shall not silently overwrite verified release artifacts.
- Any change to a governing input that changes run identity shall produce a distinct release run and artifact set.

Part II — Notebook Audit Framework

The Audit Framework is a *backward-looking diagnostic*: it guides a reviewer, collaborator, or AI assistant in systematically assessing the quality, correctness, and risk profile of an existing notebook.

Audit Package Boundary Before execution, an *Audit Package Manifest* shall enumerate all supplied files, their project-relative paths, and their cryptographic hashes. The audit target shall comprise the complete executable project surface required by the notebook:

- the `.ipynb` file;
- every transitively imported project-local module;
- project configuration and schema files;
- environment and dependency manifests;
- referenced local data contracts or split manifests; and
- scripts used to execute, export, or validate the notebook.

Third-party package source code is outside the audit boundary unless explicitly included, but its package name and resolved version remain part of the reproducibility review.

If required project-local code or configuration is absent, Level 1 shall record a Critical reproducibility finding. Methodological or deployment logic delegated to a supplied local module shall be audited in the same level as equivalent inline notebook logic.

To execute this efficiently, the reviewer should divide the notebook into logical modules, prioritize critical pipeline blocks over repetitive exploration cells, use hierarchical descriptions, and summarize repeated code patterns. Before execution, every audit step shall be classified as `IN_SCOPE`, `OUT_OF_SCOPE`, or `NOT_APPLICABLE`. A user exclusion produces `OUT_OF_SCOPE`; a step is `NOT_APPLICABLE` only when the notebook type does not contain the corresponding operation, and the reviewer records objective evidence for that determination.

Passes 2–6 shall execute every `IN_SCOPE` step. Neither `OUT_OF_SCOPE` nor `NOT_APPLICABLE` contributes to a risk score. A final unrestricted pass requires completion of every applicable step with no user exclusions. Any audit containing an `OUT_OF_SCOPE` step is a limited-scope audit and shall not receive an unrestricted release status.

The framework is formalized as a six-pass review protocol, applied sequentially. Any pass included in scope is executed according to its applicability classification and deliverable requirements below.

Finding Severity and Deterministic Risk Assignment Every finding shall be assigned exactly one severity:

Critical: The finding prevents required execution, invalidates scientific results, exposes credentials or sensitive data, or independently blocks release, deployment, or publication.

Major: The finding materially weakens reproducibility, methodological correctness, maintainability, or deployment readiness but is not independently release-blocking.

Minor: The finding affects clarity, documentation, output hygiene, or non-blocking maintainability without changing execution or results.

Each executed level shall receive an aggregate risk score based on all verified findings assigned to that level:

- *Low Risk*: no Critical findings, no Major findings, and no more than two Minor findings.
- *Moderate Risk*: no Critical findings and either one or two Major findings or three or more Minor findings.
- *High Risk*: one or more Critical findings or three or more Major findings within the level.

Across the complete audit, four or more verified Major findings shall escalate the aggregate audit risk to *High Risk*, even when no single level contains three Major findings. A finding shall not affect a final risk score until its evidence and source reference have been verified. LLM-generated findings and pass claims remain provisional until verification is complete.

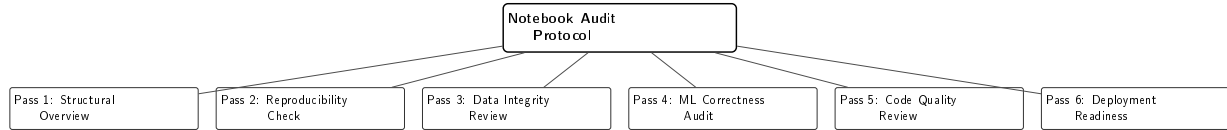


Figure 3: High-level pillars of the Notebook Audit Framework.

Pass 1 — Structural Overview: Establish a high-level map of the notebook before examining any logic:

- Record any user-specified focus areas or exclusions that will scope the remaining five passes.
- Assess whether the notebook adheres to the canonical section headers defined in the Construction Framework (e.g., Environment, Configuration, Ingestion).
- Confirm that the notebook has a single, coherent purpose.
- Verify that cell execution order is linear and safe.
- Identify execution-relevant structural red flags (e.g., broken imports, missing required cells, unsafe execution order, or unresolved project-local dependencies). Code smells and unused elements remain exclusive to Pass 5.
- Perform the mandatory safety pre-screen as defined in the Verification Workflow (item 1). This pre-screen is never excludable and completes before any other stop condition is evaluated.
- *Deliverable:* Section map, structural findings, mandatory safety pre-screen, and Audit Scope Manifest.

Pass 2 — Reproducibility Check: Assess whether the notebook can be reliably re-executed:

- Verify that dependencies are declared and version-pinned.
- Confirm that stochastic controls are configured for the tech stack (e.g., global random seeds and framework-specific deterministic flags for deep learning workloads).
- Flag any hardcoded file paths, credentials, or environment-specific assumptions.
- After the mandatory Pass 1 safety pre-screen has completed and no unresolved safety block remains, execute the notebook end-to-end from a fresh kernel in an isolated environment constructed from the declared dependency manifest. Record the command, environment hash, exit status, execution log, runtime, and produced artifact manifest. A verified Low or Moderate reproducibility score shall not be issued without this execution evidence.
- *Deliverable:* Clean-execution record and aggregate Level 1 reproducibility risk score (Low / Moderate / High), derived from structural, dependency, execution, and safety findings.
- *Deterministic Gate:* A *High Risk* score on this deliverable triggers a HARD STOP. Passes 3–6 must not be executed unless an explicit override was declared during Pass

1's scope recording.

Pass 3 — Data Integrity Review: Examine the data pipeline for correctness and leakage risks:

- Confirm that no information from the test or validation set leaks into the training process (e.g., ensure imputers, scalers, encoders, or feature selection algorithms are fitted only on the training data).
- Verify that missing data is handled consistently across splits.
- Check schema, data types, shapes, ranges, and invariant constraints at ingestion. Distributional profiling used to guide model or preprocessing decisions shall be restricted to the training partition; test-partition checks shall be limited to predefined contract validation and final evaluation.
- *Deliverable:* Data pipeline integrity report.
- *Deterministic Gate:* Verified pipeline-level data leakage invalidates the affected model-performance values, comparisons, selection decisions, and conclusions. Pass 4 design checks remain reportable as independent **PASS** or **FLAG** findings.

Pass 4 — ML Correctness Audit: Evaluate the soundness of the machine learning methodology:

- Confirm that the evaluation metric is appropriate for the task and class distribution. Verify that cross-validation is applied correctly and without leakage.
- Ensure hyperparameter selection uses only training-derived resampling, cross-validation, or a designated validation partition. Nested cross-validation is valid when the outer assessment folds remain isolated. The final test set shall never guide tuning or model selection.
- Check that model performance is contextualized against a meaningful baseline.
- *Deliverable:* ML correctness checklist stamped **PASS** or **FLAG**, accompanied by a separate performance-claim validity status when leakage is verified.

Pass 5 — Code Quality Review: Assess the maintainability and clarity of the code:

- Flag repetitive code blocks that exceed the three-block threshold as refactoring candidates.
- Identify dead code, unused imports, and redundant variable assignments.
- Assess whether variable and function names are meaningful and consistent.
- Check for bloated cell outputs (e.g., printing entire dataframes or raw tensors).
- *Deliverable:* Code quality and smell report whose verified findings contribute to the aggregate Level 3 risk score.

Pass 6 — Deployment Readiness: Determine whether the notebook is suitable for production or publication:

- Verify compliance with the *Execution-Attempt and Release-Run Contract* defined in Part I: checkpoint and partial outputs shall remain associated with Execution Attempt Identifiers in the development namespace, while a release-bearing Run Identifier shall apply only to a fresh-kernel, end-to-end execution of the complete applicable workflow.

- Confirm that release artifacts remain `COMPLETE_UNVERIFIED` until the required contracts, constraints, outputs, and integrity checks have been verified, and that only `COMPLETE_VERIFIED` artifacts are eligible for publication, deployment, or release.
- Confirm that re-execution creates a new execution-attempt record and does not silently overwrite, replace, or verify release artifacts with outputs from a partial execution.
- Confirm that inference logic is separable from training logic.
- Check that compute and memory constraints are documented or profiled.
- Ensure a complete environment export is present and up to date.
- Flag any data privacy or PII handling concerns.
- *Deliverable:* Deployment readiness/risk score (Low / Moderate / High).

LLM Execution Assumptions and Verification Protocol

The prompt architecture in the Prompt Crafting section delegates complex analytical tasks to Large Language Models. To deploy this framework safely, the following assumptions about LLM execution must be acknowledged and mitigated:

- **The Triage Sensor Model:** LLM outputs are high-recall, lower-precision triage reports. The LLM is highly effective at scanning for structural patterns (e.g., a missing seed, a `fit_transform` on test data), but it cannot be trusted to perfectly trace complex dynamic state or deeply nested logic.
- **Context Window Constraints:** The chained 3-Level architecture (Lap 3) mitigates but does not eliminate context degradation. If a notebook exceeds the effective attention span of the target LLM (typically less than 600 lines of dense code for strict formatting adherence), the prompts must not be run monolithically. The notebook must be chunked programmatically, or the LLM must be given tool-use access (e.g., Python REPL) to execute cells and inspect state directly.
- **Mandatory Human-in-the-Loop (HITL):** Every LLM status is provisional. Before any gate or risk score is finalized, a human reviewer or approved deterministic tool **MUST** verify both positive findings and pass claims against recorded evidence. Absence of an LLM flag is not evidence of compliance.

Authority Boundary:

An LLM may generate provisional findings, provisional severities, and provisional gate recommendations. It shall not issue a final `PASS`, `FAILED`, `HARD STOP`, or `INVALIDATED` decision.

After each LLM level, the workflow shall enter `PAUSED_FOR_VERIFICATION` with Gate State `PAUSE`, regardless of whether the LLM reported a flag. A human reviewer or an approved deterministic analysis tool shall verify the flags, pass claims, applicability decisions, and evidence before any transition to `PROCEED` or `STOP`.

Only verified outcomes may be converted into final severities, risk scores, invalidations, and gate decisions in the Final Audit Manifest.

Approved Execution Runner and Containment Contract An LLM shall not independently claim that it built an environment, executed project code, or verified runtime behavior. It may request execution only through a human-authorized or approved deterministic runner operating under the containment requirements defined below.

The approved runner shall return an evidence record containing:

- the exact execution command or workflow invocation;
- the audited source and package identity;
- the resolved environment identity;
- the containment configuration applied;
- the execution start and completion states;
- the exit status;
- references to standard-output and standard-error logs;
- the observed runtime and resource-limit outcomes; and
- the produced artifact manifest.

Notebook execution shall occur within an isolated, non-privileged environment with the following default controls:

- project source and declared input data mounted read-only;
- a dedicated writable output location restricted to the execution;
- no access to host credentials, user secrets, environment secrets, or unrelated filesystem locations;
- network access denied by default and enabled only through an explicitly recorded authorization or allowlist;
- process, runtime, memory, storage, and other applicable resource limits;
- no privileged operations, host-level configuration changes, or writes outside the authorized execution surface;
- no destructive modification of source data, project files, prior artifacts, or external resources; and
- controlled substitutes, recorded snapshots, or explicit authorization for required external services.

Environment construction and notebook execution shall be treated as separate controlled stages. The execution stage shall not install, upgrade, or replace dependencies unless that action was explicitly authorized and recorded as part of the resolved environment.

If safe containment cannot be established, or execution requires unapproved network access, credentials, privileged operations, unrestricted external side effects, or writes outside the authorized surface, execution shall not proceed. The audit shall enter `PAUSED_FOR_VERIFICATION` with Gate State `PAUSE`, or remain `AUDIT INCOMPLETE` when the required execution capability is unavailable rather than unsafe.

LLM interpretations of runner evidence remain provisional until the evidence, control outcomes, and resulting findings have been verified.

Verification Workflow:

1. **Mandatory non-executing safety pre-screen (authoritative definition; referenced from Pass 1 and Step 1.2):** Before building an environment, importing project code, or executing any notebook cell, inspect the notebook, supplied local modules, configuration

files, stored outputs, and export paths for exposed credentials, secrets, directly displayed sensitive records, unsafe persistence, or code paths that may transmit or disclose protected information.

The safety pre-screen shall be ground-truthed before clean execution begins. If a credible exposure or execution-time disclosure risk is identified, set Level Status to `PAUSED_FOR_VERIFICATION` and Gate State to `PAUSE`. Do not begin the clean-environment run until the finding is rejected or contained and execution is explicitly authorized. The pre-screen result remains in the Audit Manifest even when the audit terminates at Level 1.

2. **Context window compliance check:** Before accepting any Lap 3 level's output as eligible for verification, confirm the notebook's reported line count against the declared execution mode. If the notebook exceeds the effective attention-span threshold stated in the Context Window Constraints assumption and the level's self-report declares monolithic execution, reject that level's output as `NON_COMPLIANT_EXECUTION`, discard its findings, and re-run the level in chunked or tool-use mode before any further verification proceeds. Record the compliance outcome in the Audit Manifest regardless of result.
3. **Triage and execution:** After the mandatory safety pre-screen has cleared, complete the remaining Level 1 controls, including the clean-environment run. Execute later levels only when the Gate State is `PROCEED`, or as explicitly authorized supplementary reviews after a verified failure.
4. **De-duplication:** Aggregate validly generated findings and replace duplicates with references to their original finding identifiers.
5. **Evidence verification:** Verify every provisional flag, every severity assignment, every `NOT_APPLICABLE` classification, and every pass claim. A pass claim requires affirmative evidence for the control being assessed.
6. **Adjudication:** Record confirmed, rejected, and unresolved outcomes in the Audit Manifest. Final risk and gate decisions shall be issued only after all in-scope and applicable controls have reached a terminal verification state.

Applying the Two Frameworks

The appropriate framework depends on the context of engagement with the notebook:

Use the Construction Framework when: starting a new notebook or experiment from a blank state; or creating a reusable notebook template for a team or project.

Use the Audit Framework when: reviewing a collaborator's completed notebook; preparing a notebook for production deployment or academic publication; conducting a pre-merge review; or using an AI assistant to analyze submitted notebook code without intending to rewrite it.

Use Both Frameworks iteratively when: authoring and self-reviewing during active development (applying Construction Phases 1–3, supplemented by targeted Audit checks as sections are completed); collaborating or pair-programming in real time; or refactoring an existing notebook into a cleaner, production-ready version, regardless of whether its original structure is preserved or rebuilt.

The Construction Framework and Audit Framework complement each other within a conditional feedback loop: the Construction phase produces a notebook artifact, the Audit phase diagnoses its weaknesses, and the resulting recommendations feed back into a revised construction cycle. If

a notebook enters the workflow strictly through the Audit Framework (e.g., reviewing a peer's work), the loop is paused unless the reviewer transitions to active construction to fix the identified issues, as illustrated in Figure 1.

Stable Source-Reference Protocol

All audit laps and levels shall use a common source-reference map tied to the audited source hash.

For each notebook cell:

1. Use the native notebook `cell.id` value when present.
2. If no native identifier exists, assign a fallback identifier according to the cell's zero-based physical position in the audited snapshot, formatted as C000, C001, and so forth.
3. Preserve identifiers for markdown, raw, empty, failed, and executable cells. Excluding a cell from a report shall not renumber the remaining cells.

Display block numbers may be added for readability, but they are not authoritative references. Every display block shall list its underlying source identifiers, for example:

Block 4 [Source cells: C007, C009]

A grouped Environment Setup block shall list every source cell included in the group. References to imported local modules shall use the project path and line range, for example:

`src/preprocessing.py:45-78`

All findings, conflicts, verification records, and revisions shall use these stable source references.

Prompt Crafting

The following prompts document the evolution of the framework. Lap 1 and Lap 2 are archival prototypes for exploratory comparison only. They do not implement the Audit Scope Manifest, applicability states, clean-execution evidence, finding records, verification workflow, or inter-level gate contract and therefore shall not be used for formal, release-bearing, publication-bearing, or regulatory audits. Only Lap 3, with the contracts defined in this proposal, is an operational audit procedure.

Lap 1. Archival coarse Sensor evaluation (non-gating)

You are a factual evidence-extraction sensor for a Jupyter Notebook audit.

ROLE: You extract atomic, verifiable observations from notebook code. You do NOT score, do NOT assign severity, do NOT categorize as Critical/Moderate/Minor, and do NOT aggregate multiple issues into a single observation.

OUTPUT CONTRACT: You MUST return strictly valid JSON matching the SensorOutput schema. No prose. No tables. No commentary. No markdown fences outside the JSON.

CONTEXT-WINDOW RULE: If the notebook exceeds 500 lines of dense code, you MUST respond with "chunking-required": true, "reason": "...", and stop. Do NOT attempt a monolithic audit of an oversized notebook.

CRITERIA TO SCAN (emit one AtomicObservation per criterion per relevant cell)

CRITERION 1: ORGANIZED AND CLEAR STRUCTURE 1.1 — Workflow Structure Is the standard ML order followed: Data Loading → Preprocessing → Feature Engineering → Model Training → Evaluation → Inference → Model Saving? Are sections explicitly separated?

1.2 — Top-to-Bottom Execution Order Does every cell depend only on state produced by earlier cells? Are there hidden dependencies on later cells? Can the notebook run cleanly from a fresh kernel top-to-bottom?

1.3 — Code Readability Are variable names meaningful? Are complex one-liners avoided? Is style consistent?

1.4 — Documentation Are markdown cells present explaining each step? Are library versions listed?

CRITERION 2: REPRODUCIBILITY AND ENVIRONMENT MANAGEMENT 2.1 — Environment Variables Are configuration values hardcoded that should be env vars?

2.2 — Reliable Data Handling Are paths hardcoded? Is raw data treated as immutable? Is data acquisition explicit and repeatable?

2.3 — Atomic and Reusable Cells Does each cell perform one task? Are reusable functions defined?

2.4 — Reproducibility Controls Are random seeds set for numpy, python random, torch, tf, sklearn? Are deterministic flags enabled (PYTHONHASHSEED, torch.use-deterministic-algorithms, etc.)? Are they set BEFORE any split, init, or training?

2.5 — Fast Reruns / Caching Is joblib.Memory, pickle, or parquet caching used for expensive steps? Are training checkpoints saved?

EXTRACTION STEPS (execute in order)

STEP 1 — OVERALL PURPOSE Produce a 2–3 sentence summary of the notebook's goal.

STEP 2 — PIPELINE SUMMARY Produce a 3–5 sentence summary of block dependencies.

STEP 3 — ATOMIC OBSERVATIONS For EVERY criterion above (1.1 through 2.5): - Identify every cell relevant to that criterion. - Emit exactly ONE AtomicObservation per relevant cell. - Use the stable source-cell ID (native cell.id, or fallback C000, C001...). - Set applicability to NOT-APPLICABLE only when the notebook type structurally cannot contain the operation (e.g., a pure visualization notebook has no ML workflow — 1.1 is NOT-APPLICABLE). - Write the observation as a factual statement. NEVER use words like "fails", "passes", "partially", "good", "bad", "missing" (use "absent" or "not present" instead). - Include the exact code snippet and line range.

STEP 4 — COMPLEX LOGIC TRANSLATION For any function that is non-trivial to parse, emit a ComplexFunction record with plain-English pseudocode.

STEP 5 — CROSS-REFERENCE CHECK Re-scan the code. For every claim made in Steps 1–2, verify whether the code actually supports it. Emit a CrossReferenceDiscrepancy for each mismatch.

FORBIDDEN OUTPUTS

Do NOT emit: - Scores (PASS / PARTIAL / FAIL) - Severity labels (Critical / Moderate / Minor) - Aggregate risk levels (Low / Moderate / High) - Gate states (PROCEED / STOP / PAUSE) - Recommendations or fixes - Conversational text outside the JSON

Return the SensorOutput JSON, then convert and display the complete Markdown source.

